CSEP 590tv: Quantum Computing

Dave Bacon

Aug 3, 2005

Today's Menu

Administrivia
Public Key Cryptography
Shor's Algorithm
Grover's Algorithm
Entanglement



Peter Shor

Administrivia

Hand in HW #5.

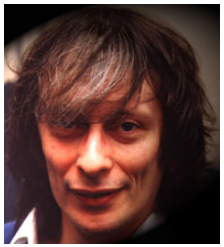
Pick up HW solutions.

Pick up the Take Home Final!

Two weeks to complete. No collaboration.

Extra credit problem based on next week's lecture on entanglement.

Review



David Deutsch



Richard Jozsa

1992: Deutsch-Jozsa Algorithm

Exact classical q. complexity: $2^{n-1}+1$

Bounded error classical q. complexity: $O(1)$

Exact quantum q. complexity: 1



Umesh Vazirani



Ethan Bernstein

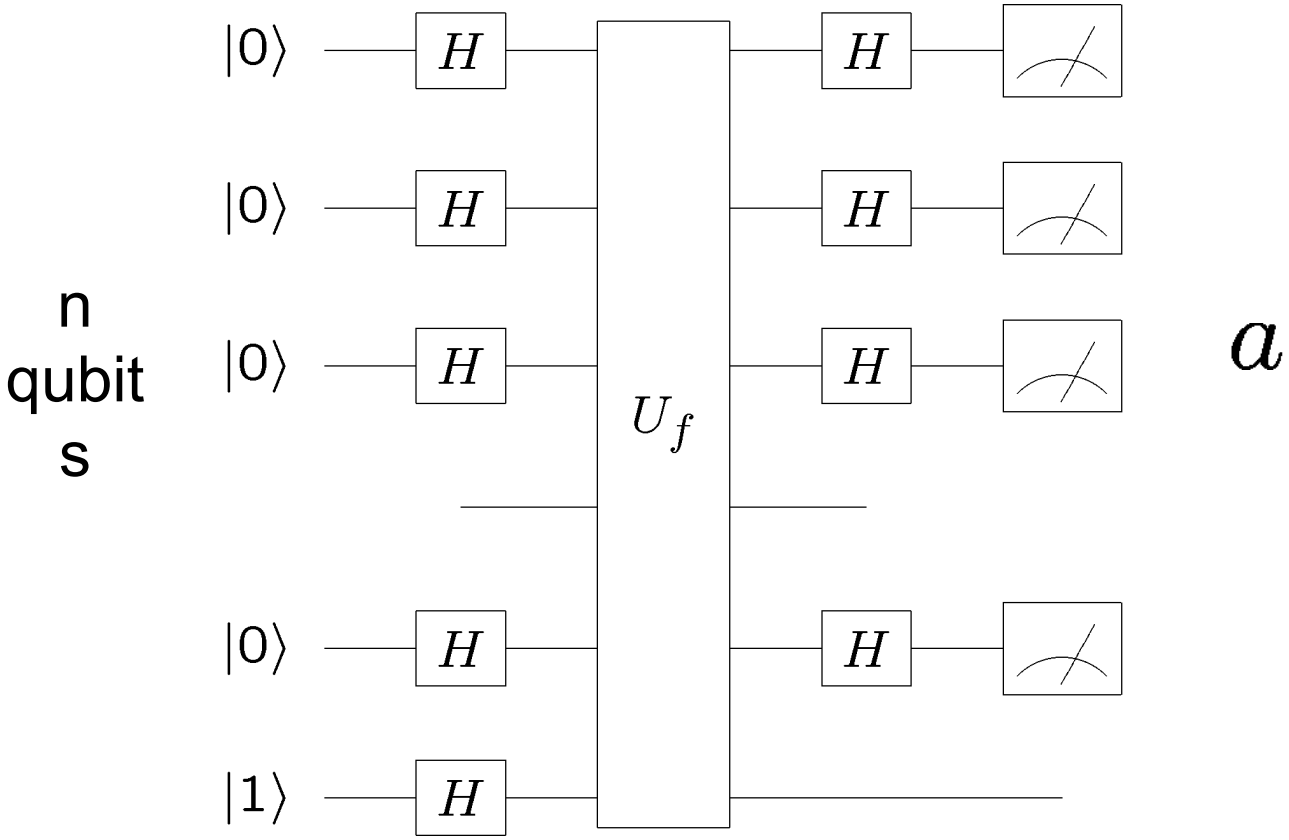
1993: Bernstein-Vazirani Algorithm (non-recursive)

Exact classical q. complexity: n

Bounded error classical q. complexity: $\Omega(n)$

Exact quantum q. complexity: 1

Review



Deutsch-Jozsa Algorithm
Bernsetein-Vazirani Algorithm

Review



Dan Simon

1994: Simon's Algorithm

Bounded error classical q. complexity: $\Omega(2^{n/2})$

Bounded error quantum q. complexity: $O(n)$

(first exponential separation)

Given: A function with n bit strings as input and one bit as output

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}^n$$

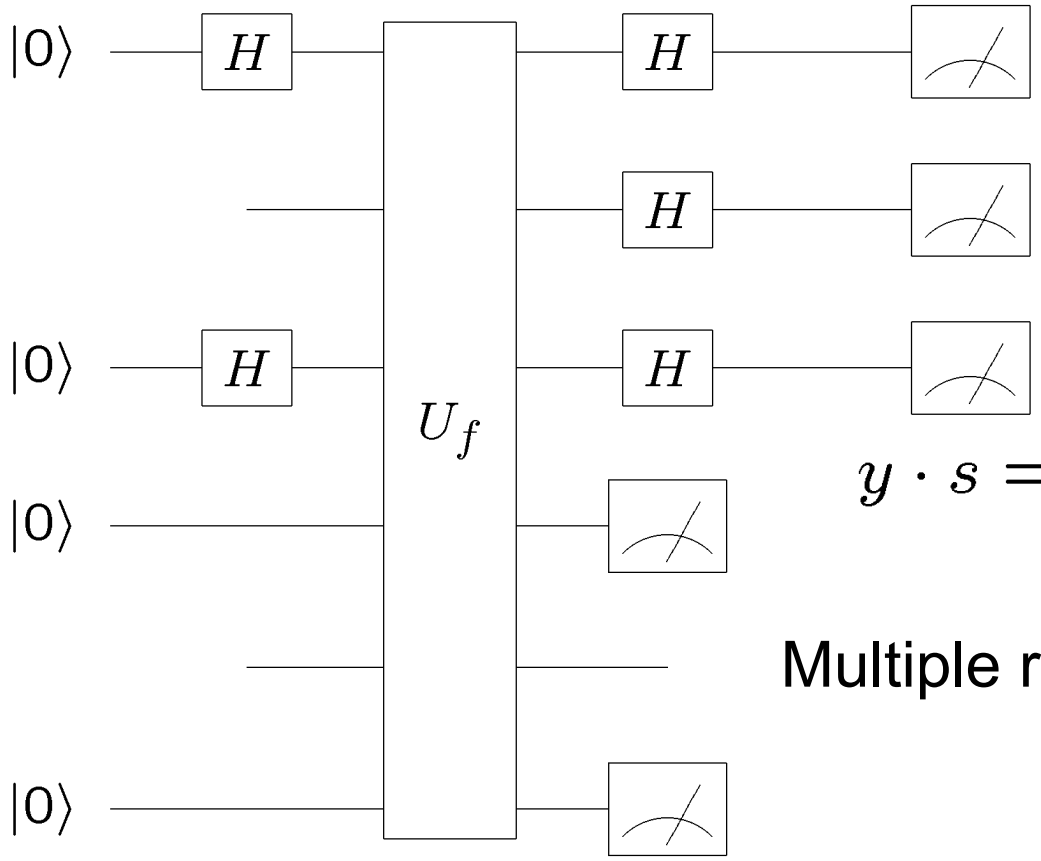
Promise: The function is guaranteed to satisfy

$$f(x) = f(y) \Leftrightarrow y = x \oplus s, s \neq 0$$

$$x \oplus s = (x_1 \oplus s_1, x_2 \oplus s_2, \ldots, x_n \oplus s_n)$$

Problem: Find the n bit string $s \neq 0$

Review



Simon's algorithm

Dan Simon - "You know, I really apologize: I was young, needed the money, I just had to do that..."

APS Journals Physical Review Letters
AMERICAN PHYSICAL SOCIETY

Fast Quantum Algorithm for Numerical Gradient Estimation

Stephen P. Jordan*

Physics Department, MIT, 77 Massachusetts Avenue, Cambridge, Massachusetts 02139, USA
(Received 25 May 2004; revised manuscript received 3 January 2005; published 28 July 2005)

Given a black box for f, a smooth real scalar function of d real variables, one wants to estimate ∇f at a given point with n bits of precision. On a classical computer this requires a minimum of $d+1$ black box queries, whereas on a quantum computer it requires only one query regardless of d. The number of bits of precision to which f must be evaluated matches the classical requirement in the limit of large n.

DOI: 10.1103/PhysRevLett.95.050501 PACS numbers: 03.67.Lx

We investigate the query complexity of numerically estimating the gradient of a black box function $f: R^d \rightarrow R$ at a given point. We find that gradients can be estimated on a quantum computer using a single black box query whereas we require at least $d+1$ queries classically. The algorithm which achieves this can be viewed as a generalization of the Bernstein-Vazirani [1] algorithm, which has been described in other contexts [2–5]. The black box in this algorithm was always previously described as evaluating a function over the integers rather than approximating a continuous function with finite precision. In [2], the question as to whether the algorithm could be adapted for any task of practical interest was presented as an open problem, which this paper resolves by showing a speedup for gradient estimation, which is a fundamental operation in many numerical calculations.

Today:
Factoring

One Time Pads

Random n bit string

0010101111010001



Alice



Bob

Alice's message

0110110011100101

secret key

$\oplus$

0100011100110100

secret key

$\oplus$

0110110011100101

Eve cannot learn message

Public Key Cryptography

Interesting history:

1st schemes “known in public” where put forth by
Diffie and Hellman in 1976 (key exchange) and
Rivest, Shamir and Adleman in 1978 (encryption algorithm)
(based on work by Merkle in 1974, published 1978)

However, it now appears that the British researchers working for British intelligence (GCHQ) were actually the first to discover these protocols, but their work was classified at the time!
Clifford Cooks in 1973 (encryption algorithm)
Malcolm Williamson in ~1973 (key exchange)
(based on work by James Ellis in the late 1960s.)

Computational Complexity

P : decision problems which can be solved without error in polynomial time on a deterministic classical Turing machine.

Decision problems: problem with a yes/no answer.

Polynomial time: worst case bounded by a polynomial in the size of the problem.

Examples of problems in P:

Perfect matching: does a given graph have a perfect matching?

Primes: is a given number a prime number?

Linear Equalities: Given an integer n x d matrix A and an integer n x 1 vector b, does there exists a rational d x 1 vector x>0 such that Ax=b?

Computational Complexity

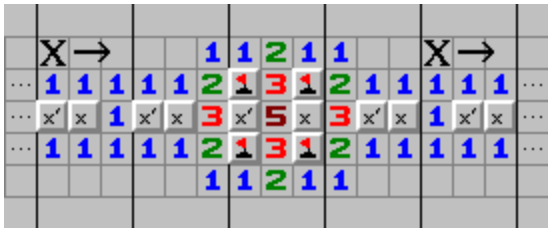
NP : decision problems which can be solved without error in a polynomial time on a classical nondeterministic Turing machine. Shorthand, decision problems which, given a solution, and you can verify this solution in polynomial time on a deterministic classical Turing machine.

Examples of problems in NP:

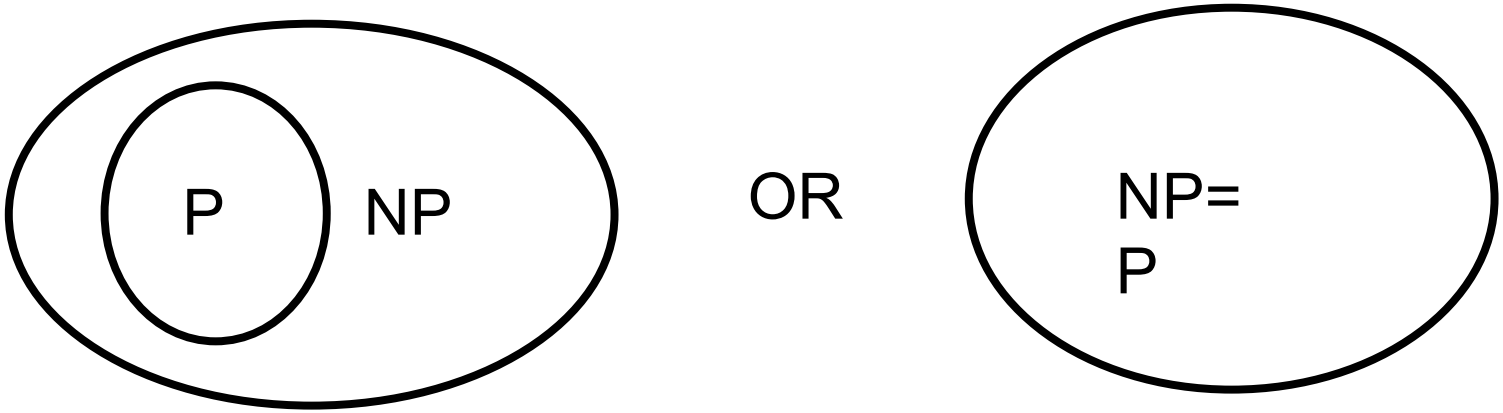
Perfect matching: does a given graph have a perfect matching?

Satisfaction: does a given boolean function have a satisfying assignment? Given f(x1,x2,…,xn), does there exist x={0,1}n such that f(x)=1?

Minesweeper: Given a partially solved Minesweeper board, does there exist an assignment of mines which can give rise to this board?

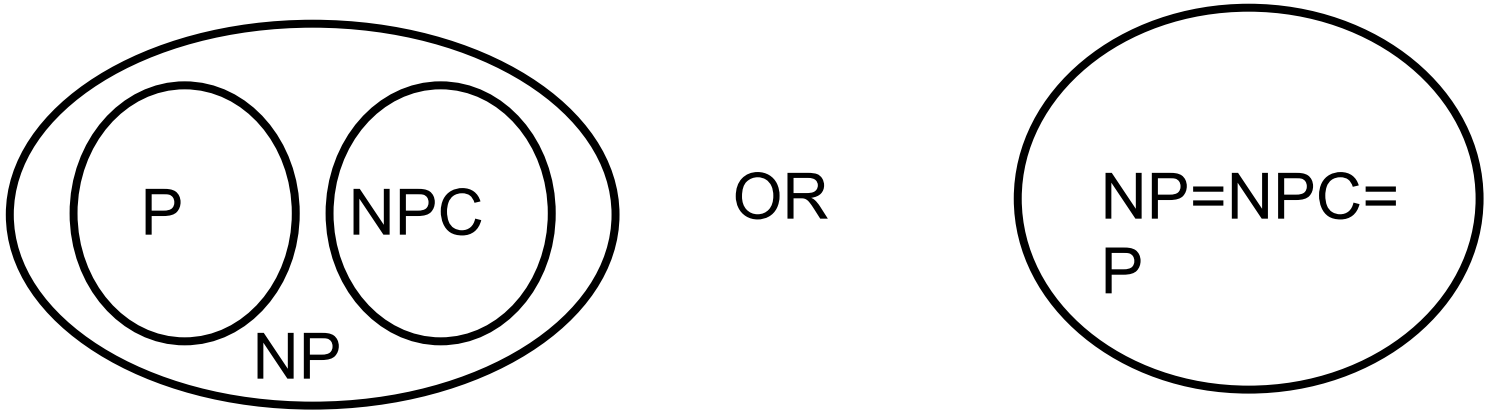


One Million Dollars



NP – Hard: Problems which have the property that for every problem in NP there is a polynomial time reduction to this problem.

NP – Complete (NPC): NP – Hard and in NP.



Public Key Cryptography

1. There probably exist computational problems that are HARD.

2. Can we use these to perform secure cryptography by basing the security of the problem on the difficulty of the hard problem?

If we make the hard problem big enough, baring a breakthrough in the computational complexity of the problem, or in computer hardware technology, the cryptography will be secure

Public Key Cryptography Roughly



Alic

Instructions for how to make her lock.



Bob

Bob's secret documents





Assume: very hard to design key from instructions to make lock

This is (very roughly) what happens in public key cryptography

Public Key Encryption RSA



Alic
e



Bob

1. Alice generates two random large primes p q and $N = pq, \phi = (p-1)(q-1)$
2. Alice chooses a number $1 < e < \phi$ which is coprime with ϕ.
3. Alice computes d such that $de = 1 \mod \phi$

Public Key: (N, e)

Private Key: (N, d)

Public Key Encryption: RSA



Public Key: (N, e)



Alic
e

Bob

Public Key: (N, e)

Private Key: (N, d)

Bob's message: n

$$c = n^e \bmod N$$

$$c = n^e \bmod N$$

$$c^d \bmod N = n^{ed} \bmod N$$

$$= n \bmod p = n \bmod q \quad \text{(FLT)}$$

$$= n \bmod pq = n \bmod N \quad \text{(CRT)}$$

Public Key Encryption: RSA



Alic
e

Public Key: (N, e)

Private Key: (N, d)



Bob

Bob's message: n

Bob, using public key can encrypt message.

Alice, using private key, can decrypt the message

But decrypting without the private key is (thought) to be computationally hard

Public Key Encryption: RSA



Alic
e

Public Key: (N, e)

Private Key: (N, d)



Bob

Bob's message: n

<u>Factoring can be used to break RSA</u>

If we could factor, $N = pq$ then we could compute $\phi = (p-1)(q-1)$ from which you could use e to find d

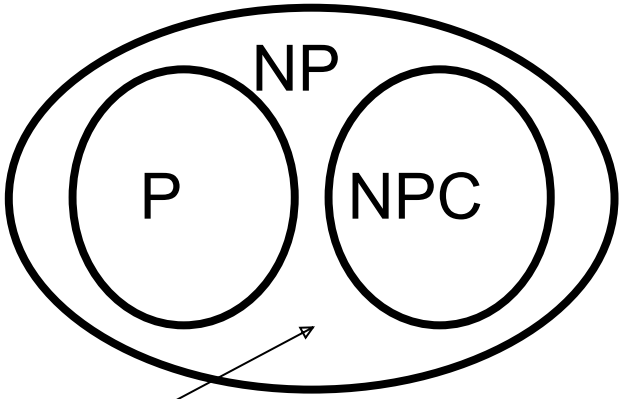
$$de = 1 \mod \phi$$

Then we just use the standard decryption:

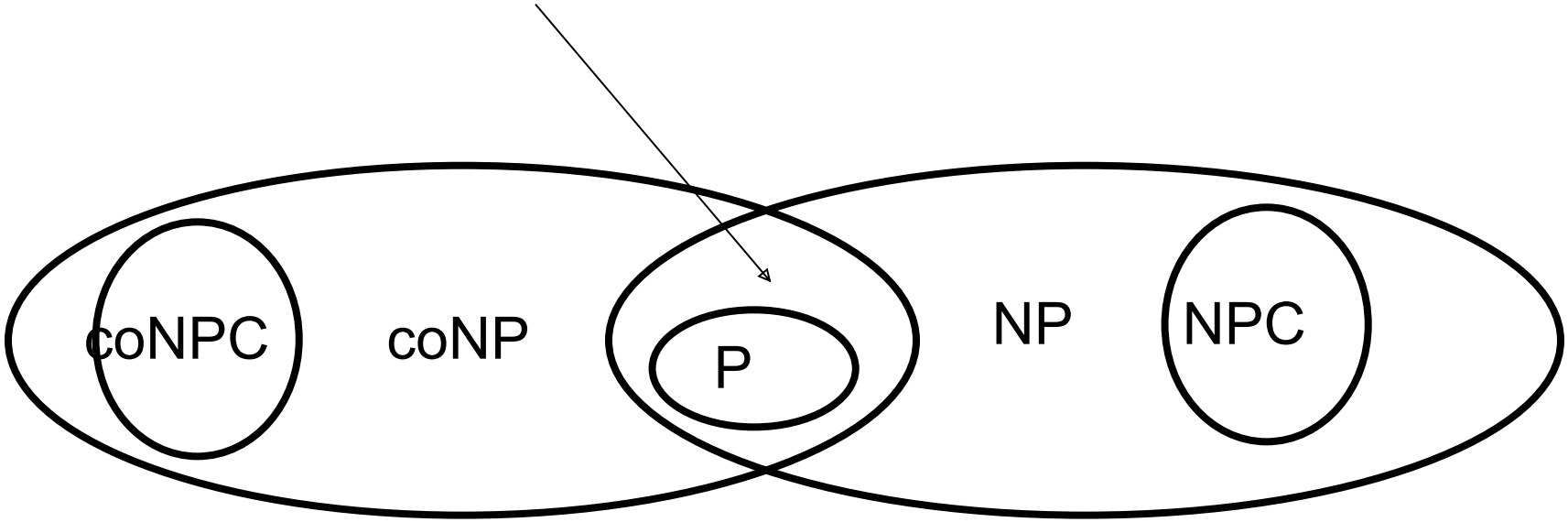
$$c^d \mod N = n \mod N$$

Factoring

Difficulty?
Probably:



Factoring: Is one of the factors less than k?



coNP: efficiently verifiable that NO solution to problem exists.

Shor's Algorithm



Peter Shor 1994

188198812920607963838697239461650439807163563379417382700763356422988859715234665485319060606504743045317388011303396716199692321205734031879550656996221305168759307650257059

=

39807508642406493739712550055038649119906436234252670840638518957594638895726176858331794727721461074353025362230719730482246329146953020971164598521711305207112563635903975271

×

Best classical algorithm takes time $O(\exp(n^{1/3}))$

Shor's quantum algorithm takes time $O(n^3 \log n)$

Shor's Algorithm

What were the key insights which Shor used?

Simon's problem work's because the function has a symmetry:

$$f(x) = f(y) \Leftrightarrow y = x \oplus s$$

In this case the symmetry is a $\mathbb{Z}_2^n$ symmetry

Shor became interested in different symmetries and in particular symmetries of $\mathbb{Z}_N = \{0, 1, \ldots, N-1\}$

"the place where we do addition modulo N"

Period Finding

Given: A function from 0,1,…,N-1 to some n bit numbers

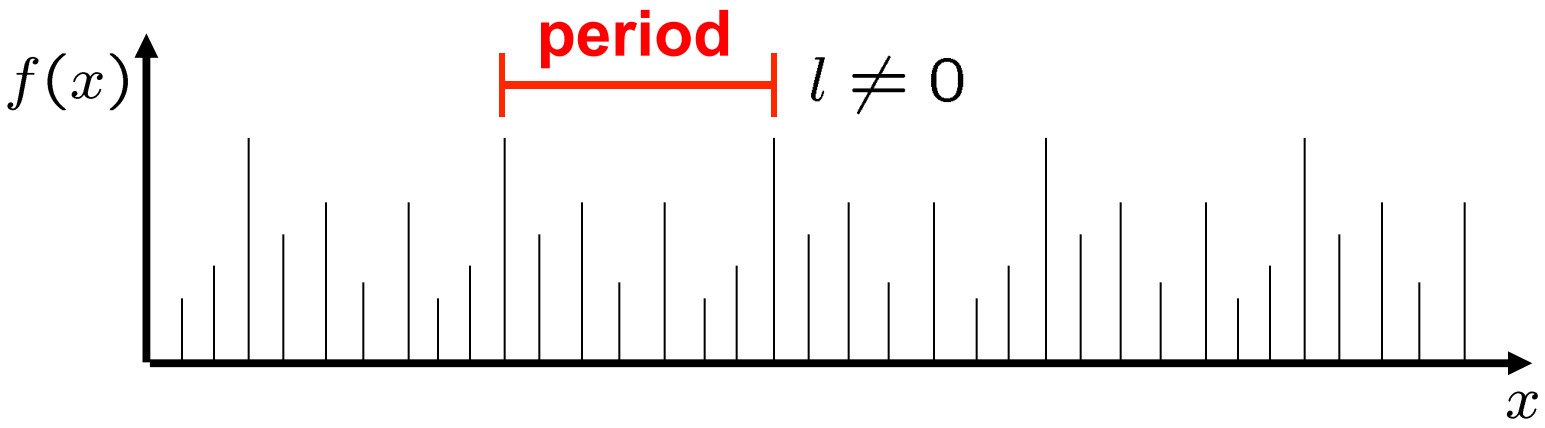
$$f : x \in \{0, 1, 2, \ldots, N-1\} \rightarrow \{0,1\}^n$$

Promise: The function is guaranteed to satisfy

$$f(x) = f(y) \Leftrightarrow y = x + l \qquad l \neq 0$$

Problem: Find the hidden period $l \neq 0$

$$\{0, 1, 2, \ldots, N-1\}$$



Shor's Algorithm

What were the key insights which Shor used?

1. Period finding
2. Period finding can be perform efficiently on a quantum computer.
3. Period finding can be used to factor integers

Order-Finding and Factoring

Factor *N*

choose *x* coprime to *N* (Euclid's algorithm for gcd)

Order finding: find smallest *r* such that $x^r = 1 \mod N$

Use order finding to factor: suppose r is even,

$x^r - 1 = 0 \mod N$ N divides $x^r - 1$

N divides $(x^{\frac{r}{2}} - 1)(x^{\frac{r}{2}} + 1)$

But $1 \leq x^{\frac{r}{2}} \leq N$ N must share a common factor N with $x^{\frac{r}{2}} \pm 1$ not equal to

If *r* is even then compute $\gcd(x^{r/2} \pm 1, N)$ as factor!

More tricky: r is even happens with high probability

Order-Finding and Period-Finding

Order finding: find *r* such that $x^r = 1 \mod N$

Find the period of $f(s) = x^s \mod N$

What were the key insights which Shor used?

1. Period finding
2. Period finding can be perform efficiently on a quantum computer.
3. Period finding can be used to factor integers

To understand period finding, we need to understand
Fourier transforms

Fourier Transforms

Function of a single bit: $f(0), f(1)$

We could equally well deal with

$$g(0) := f(0) + f(1)$$
$$g(1) := f(0) - f(1)$$

Because we can "invert"

$$f(0) = \frac{1}{2}(g(0) + g(1))$$
$$f(1) = \frac{1}{2}(g(0) - g(1))$$

"Look" familiar?

Fourier Transforms

$$|v\rangle = \alpha|0\rangle + \beta|1\rangle \quad \boxed{H} \quad = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

Output:

$$|v'\rangle = H|v\rangle = \alpha\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + \beta\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

$$g(0) := f(0) + f(1)$$

$$g(1) := f(0) - f(1)$$

$$= \frac{\alpha+\beta}{\sqrt{2}}|0\rangle + \frac{\alpha-\beta}{\sqrt{2}}|1\rangle$$

The Hadmard is performing this transform (up to a constant) on the AMPLITUDES of our wave function!

Fourier Transforms

Function on N different inputs: $f(0), f(1), \ldots, f(N-1)$

We can the define the following N new numbers to represent the function:

$$\hat{f}(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{xy} f(x)$$

Slow down there egghead….

Nth root of unity: $\omega_N = \exp\left(\frac{2\pi i}{N}\right)$

Nth Root Of

$$\omega_N = \exp\left(\frac{2\pi i}{N}\right) = \cos\left(\frac{2\pi}{N}\right) + i\sin\left(\frac{2\pi}{N}\right)$$

Unit modulus:

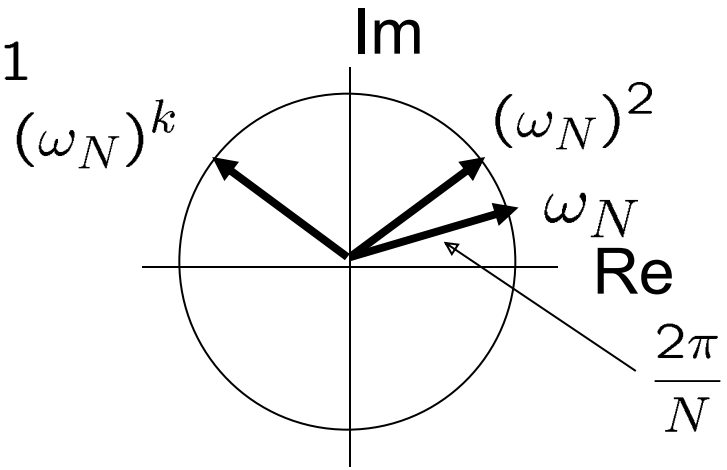
$$|\omega_N|^2 = \omega_N^* \omega_N = \exp\left(-\frac{2\pi i}{N}\right)\exp\left(\frac{2\pi i}{N}\right)\exp\left(-\frac{2\pi i}{N} + \frac{2\pi i}{n}\right)$$

$$= \exp(0) = 1$$

Nth root of unity:

$$(\omega_N)^N = \exp\left(\frac{2\pi i}{N}\right)^N = \exp\left(\frac{2\pi i}{N}N\right) = e^{2\pi i} = 1$$

$$(\omega_N^j)^N = \omega_N^{jN} = 1$$



Nth Root Of

$$\omega_N = \exp\left(\frac{2\pi i}{N}\right) = \cos\left(\frac{2\pi}{N}\right) + i\sin\left(\frac{2\pi}{N}\right)$$

The big sum:

$$\begin{aligned} S = \sum_{k=0}^{N-1} r^k &= 1 + r + r^2 + \cdots + r^{N-1} \\ &= 1 + r(1 + r + r^2 + \cdots + r^{N-1}) - r^N \\ &= 1 + rS - r^N \end{aligned}$$

$$(1-r)S = 1 - r^N \qquad S = \frac{1-r^N}{1-r} \quad \text{for } r \neq 1$$

$$S = N \quad \text{for } r = 1$$

$$\sum_{k=0}^{N-1} (\omega_N)^{jk} = \sum_{k=0}^{N-1} (\omega_N^j)^k = \frac{1-(\omega_N^j)^N}{1-\omega_N^j} = 0$$

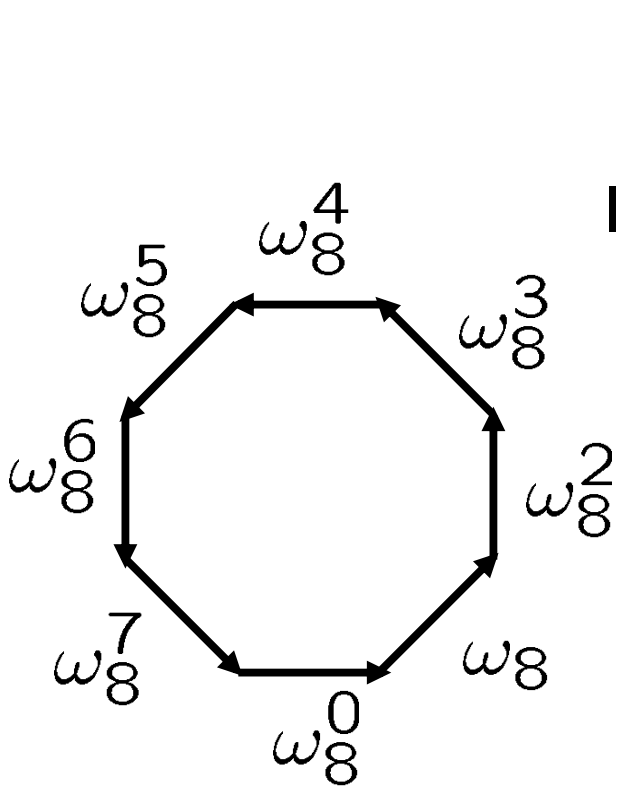
Unless $j = 0$ and then $\sum_{k=0}^{N-1} (\omega_N)^0 = N$

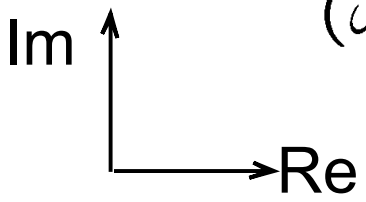
Nth Root Of

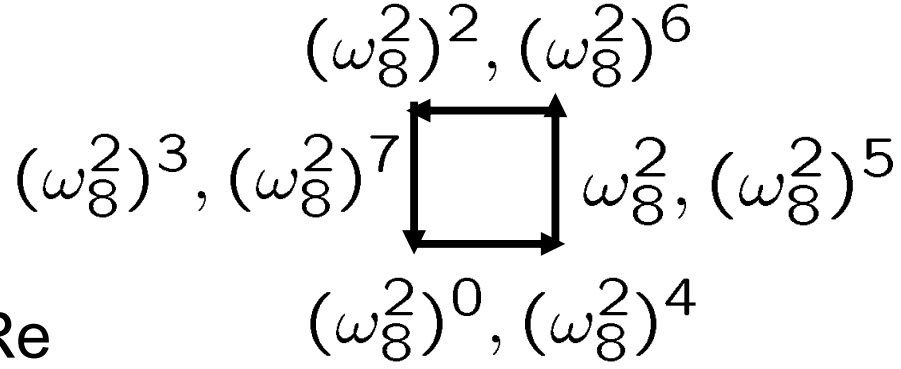
$$\omega_N = \exp\left(\frac{2\pi i}{N}\right) = \cos\left(\frac{2\pi}{N}\right) + i\sin\left(\frac{2\pi}{N}\right)$$

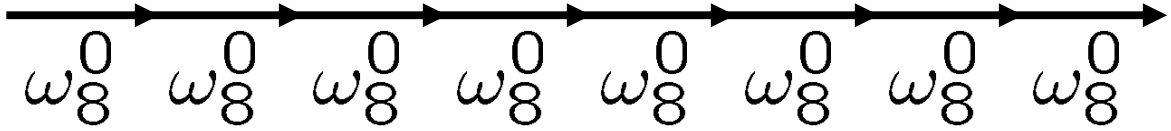
The sum of all sums:

$$\frac{1}{N}\sum_{k=0}^{N-1}(\omega_N)^{jk} = \delta_{j,0}$$









Fourier Transforms

Function on N different inputs: $f(0), f(1), \ldots, f(N-1)$

We can the define the following N new numbers to represent the function:

$$\hat{f}(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{xy} f(x)$$

$$\hat{f}(0), \hat{f}(1), \ldots, \hat{f}(N-1)$$

Now we can see how to go from the hats back to the non hats!

$$f(x) = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} \hat{f}(y)$$

Fourier Transforms

$f(0), f(1), \ldots, f(N-1)$ $\qquad$ $\hat{f}(0), \hat{f}(1), \ldots, \hat{f}(N-1)$

$$f(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{-xy} \hat{f}(x) \qquad \hat{f}(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{xy} f(x)$$

It works!....

$$f(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{-xy} \frac{1}{\sqrt{N}} \sum_{x'=0}^{N-1} \omega_N^{x'x} f(x') = \sum_{x'=0}^{N-1} \frac{1}{N} \sum_{x=0}^{N-1} \omega_N^{x'x-xy} f(x')$$

$$= \sum_{x'=0}^{N-1} \delta_{y,x'} f(x') = f(y)$$

$$\delta_{x'-y,0} = \delta_{x',y}$$

Fourier Transforms

Example $N = 3$:

$$\hat{f}(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{xy} f(x)$$

$$\hat{f}(0) = \frac{1}{\sqrt{3}} \left[\omega_3^{0 \cdot 0} f(0) + \omega_3^{0 \cdot 1} f(1) + \omega_3^{0 \cdot 2} f(2) \right]$$

$$\hat{f}(1) = \frac{1}{\sqrt{3}} \left[\omega_3^{1 \cdot 0} f(0) + \omega_3^{1 \cdot 1} f(1) + \omega_3^{1 \cdot 2} f(2) \right]$$

$$\hat{f}(1) = \frac{1}{\sqrt{3}} \left[\omega_3^{2 \cdot 0} f(0) + \omega_3^{2 \cdot 1} f(1) + \omega_3^{2 \cdot 2} f(2) \right]$$

$$\hat{f}(0) = \frac{1}{\sqrt{3}} \left[f(0) + f(1) + f(2) \right]$$

$$\hat{f}(1) = \frac{1}{\sqrt{3}} \left[f(0) + \omega_3 f(1) + \omega_3^2 f(2) \right]$$

$$\hat{f}(2) = \frac{1}{\sqrt{3}} \left[f(0) + \omega_3^2 f(1) + \omega_3 f(2) \right]$$

Fourier Transforms

$$\hat{f}(0) = \frac{1}{\sqrt{3}}[f(0) + f(1) + f(2)]$$

$$\hat{f}(1) = \frac{1}{\sqrt{3}}\left[f(0) + \omega_3 f(1) + \omega_3^2 f(2)\right]$$

$$\hat{f}(2) = \frac{1}{\sqrt{3}}\left[f(0) + \omega_3^2 f(1) + \omega_3 f(2)\right]$$

Example

:

$$(f(0), f(1), f(2)) = (1, 0, i)$$

Fourier transformed coefficients:

$$(\hat{f}(0), \hat{f}(1), \hat{f}(2)) = (\frac{1}{\sqrt{3}}(1+i), \frac{1}{\sqrt{3}}(\omega_3 + i\omega_3^2), \frac{1}{\sqrt{3}}(\omega_3 + i\omega_3))$$

Unitarity & Fourier Transforms

$$|v\rangle = \alpha|0\rangle + \beta|1\rangle \quad \boxed{H} \quad = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

Output:

$$|v'\rangle = H|v\rangle = \alpha\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + \beta\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

$$= \frac{\alpha+\beta}{\sqrt{2}}|0\rangle + \frac{\alpha-\beta}{\sqrt{2}}|1\rangle$$

$$f(0) = \alpha \qquad \hat{f}(0) = \frac{\alpha+\beta}{\sqrt{2}}$$

$$f(1) = \beta \qquad \hat{f}(1) = \frac{\alpha-\beta}{\sqrt{2}}$$

$$f(0)|0\rangle + f(1)|1\rangle \quad \boxed{H} \quad \hat{f}(0)|0\rangle + \hat{f}(1)|1\rangle$$

New amplitudes are Fourier transform of old amplitudes!

Quantum Fourier Transform

The quantum Fourier transform:

$$U_{QFT} = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{xy} |y\rangle\langle x|$$

See it in action:

$$|\psi\rangle = \sum_{x=0}^{N-1} f(x)|x\rangle \qquad \langle a|\psi\rangle = f(a)$$

$$U_{QFT}|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{xy} |y\rangle\langle x|\psi\rangle$$

$$U_{QFT}|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{xy} f(x)|y\rangle = \sum_{y=0}^{N-1} \hat{f}(y)|y\rangle$$

Quantum Fourier Transform

The quantum Fourier transform:

$$U_{QFT} = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{xy} |y\rangle\langle x| \qquad U_{QFT}^{\dagger} = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{-xy} |x\rangle\langle y|$$

But is it unitary?

$$U_{QFT} U_{QFT}^{\dagger} = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{xy} |y\rangle\langle x| \frac{1}{\sqrt{N}} \sum_{y'=0}^{N-1} \sum_{x'=0}^{N-1} \omega_N^{-x'y'} |x'\rangle\langle y'|$$

$$= \frac{1}{N} \sum_{y,y'=0}^{N-1} \sum_{x,x'=0}^{N-1} \omega_N^{xy-x'y'} |y\rangle\langle x|x'\rangle\langle y'|$$

$$= \frac{1}{N} \sum_{y,y'=0}^{N-1} \sum_{x,x'=0}^{N-1} \omega_N^{xy-x'y'} \delta_{x,x'} |y\rangle\langle y'| = \frac{1}{N} \sum_{y,y'=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{x(y-y')} |y\rangle\langle y'|$$

$$= \sum_{y,y'=0}^{N-1} \delta_{y,y'} |y\rangle\langle y'| = \sum_{y=0}^{N} |y\rangle\langle y| = I \qquad U_{QFT} U_{QFT}^{\dagger} = I$$

Quantum Fourier Transform

And about that inverse QFT:

$$U_{QFT}^{\dagger} = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{-xy} |x\rangle\langle y|$$

$$|\widehat{\psi}\rangle = \sum_{x=0}^{N} \widehat{f}(x)|x\rangle$$

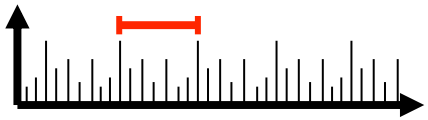
$$U_{QFT}|\widehat{\psi}\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{xy} |y\rangle\langle x|\widehat{\psi}\rangle$$

$$U_{QFT}|\widehat{\psi}\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} \omega_N^{-xy} \widehat{f}(x)|y\rangle = \sum_{y=0}^{N} f(y)|y\rangle$$

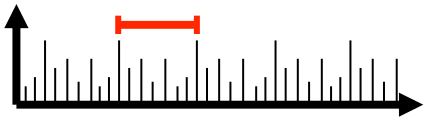
$$f(y) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \omega_N^{-xy} \widehat{f}(x)$$

It performs the inverse Fourier transform on the amplitudes!

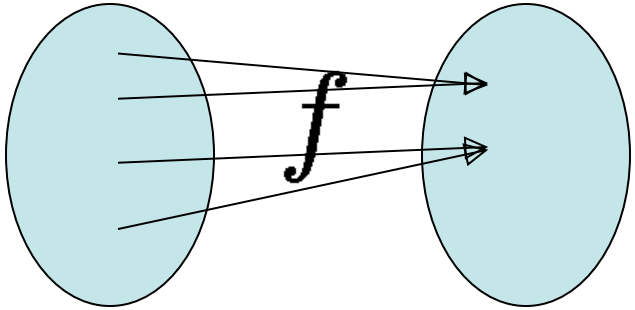
In Class Problem #1



Period Finding



$x \in \{0,1\}^n \qquad y \in \{0,1\}^n$

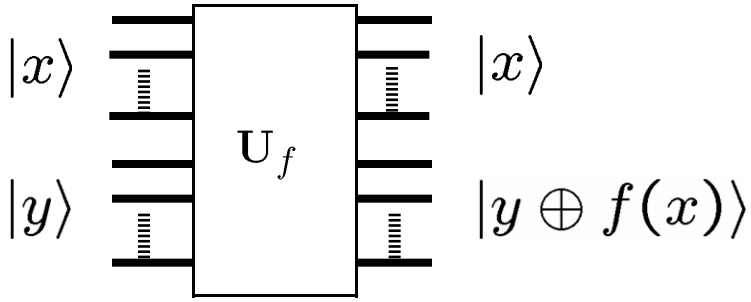


$$f(x) = f(x+l)$$

Problem: find l in as few queries as possible

Period Finding Problem

quantum oracle



$x, y, f(x) \in \{0,1\}^n$

….in as few uses of the quantum oracle as possible

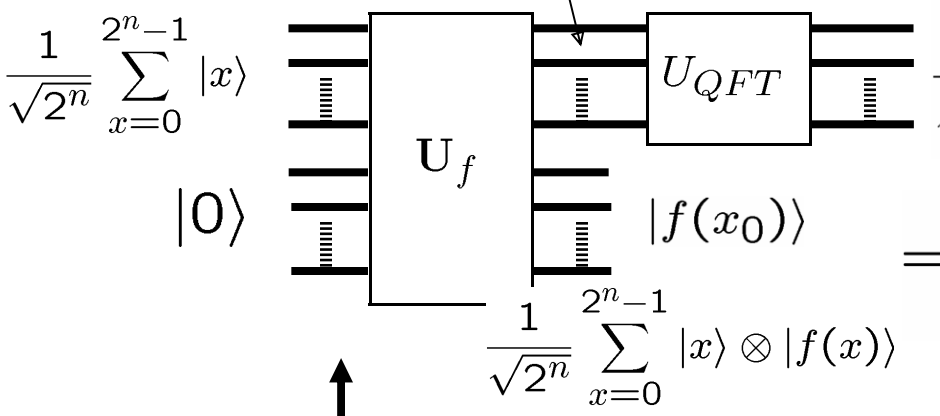
a symmetric problem!

Fourier to the Rescue



$$U_{QFT} = \frac{1}{\sqrt{2^n}} \sum_{x,p=0}^{2^n-1} \exp\left(\frac{2\pi i p x}{2^n}\right) |p\rangle\langle x|$$

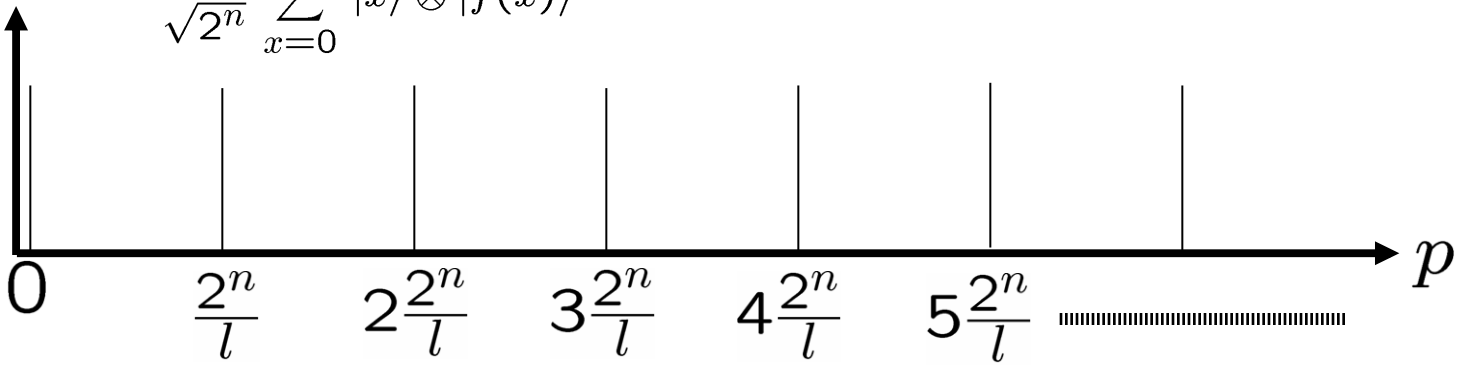
$$|\psi(x_0)\rangle = \frac{1}{\sqrt{2^n/l}} \sum_{j=0}^{2^n/l-1} |x_0 + jl\rangle$$



$$\frac{1}{\sqrt{2^{2n}/l}} \sum_{p=0}^{2^n-1} \sum_{j=0}^{2^n/l-1} \exp\left(\frac{2\pi i p(x_0 + jl)}{2^n}\right) |p\rangle$$

$$= \frac{1}{\sqrt{l}} \sum_{p=0|pl=0 \bmod 2^n}^{2^n-1} e^{\frac{2\pi i p x_0}{2^n}} |p\rangle$$

probability



Shor's Algorithm

To Factor N on a quantum computer:

Select *x* coprime to *N*

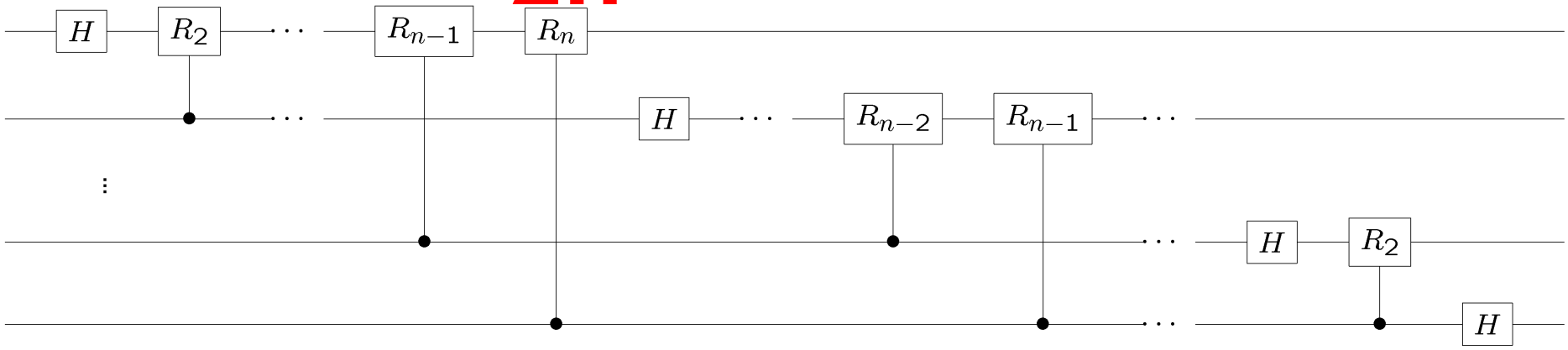
Use the quantum computer to find the period of

$$f(s) = x^s \bmod N$$

Use order of *x* to compute possible factors of N.
Check if they work. If not rerun.

Running time? How many quantum gates?

QFT over 2n



$$H = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \qquad R_k = \begin{bmatrix} 1 & 0 \\ 0 & \omega_{2^k} \end{bmatrix}$$

This circuit requires O(n2) “elementary” gates

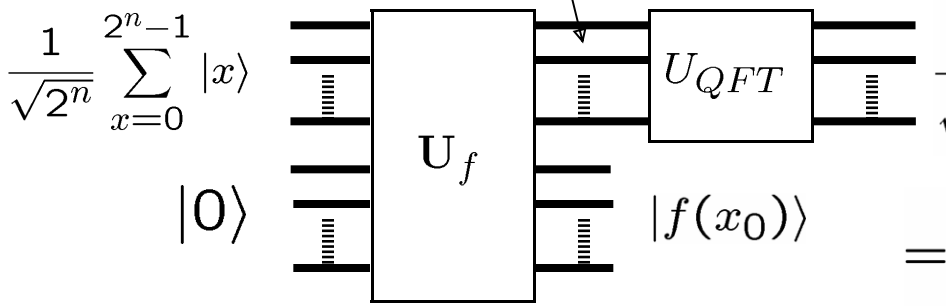
QFTs for all other Ns can similarly be implemented.

Fourier to the Rescue



$$U_{QFT} = \frac{1}{\sqrt{2^n}} \sum_{x,p=0}^{2^n-1} \exp\left(\frac{2\pi i p x}{2^n}\right) |p\rangle\langle x|$$

$$|\psi(x_0)\rangle = \frac{1}{\sqrt{2^n/l}} \sum_{j=0}^{2^n/l-1} |x_0 + jl\rangle$$



$$\frac{1}{\sqrt{2^{2n}/l}} \sum_{p=0}^{2^n-1} \sum_{j=0}^{2^n/l-1} \exp\left(\frac{2\pi i p(x_0 + jl)}{2^n}\right) |p\rangle$$

$$= \frac{1}{\sqrt{l}} \sum_{p=0|pl=0 \bmod 2^n}^{2^n-1} e^{\frac{2\pi i p x_0}{2^n}} |p\rangle$$

$$f(s) = x^s \bmod N$$

O(n3) “elementary” gates

modular exponentiation

Shor's Algorithm

To Factor N on a quantum computer:

Select *x* coprime to *N*

Use the quantum computer to find the period of

$$f(s) = x^s \bmod N$$

Use order of *x* to compute possible factors of N.
Check if they work. If not rerun.

Running time:
O(n3)

Shor's Algorithm



Peter Shor 1994

188198812920607963838697239461650439807163563379417382700763356422988859715234665485319060606504743045317388011303396716199692321205734031879550656996221305168759307650257059

=

39807508642406493739712550055038649119906436234252670840638518957594638895726176858331794727721461074353025362230719730482246329146953020971164598521711305207112563635903975271

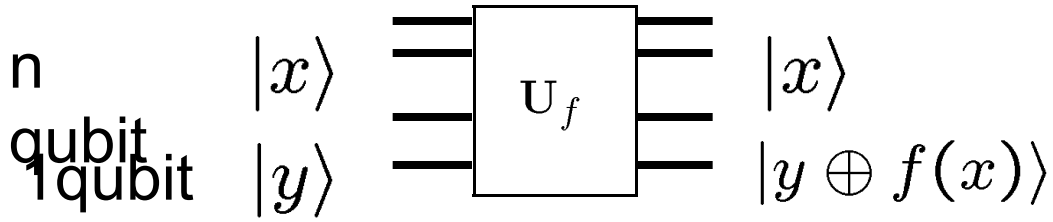
×

Best classical algorithm takes time $O(\exp(n^{1/3}))$

Shor's quantum algorithm takes time $O(n^3 \log n)$

Grover's Problem

Suppose we have a black box



with the property

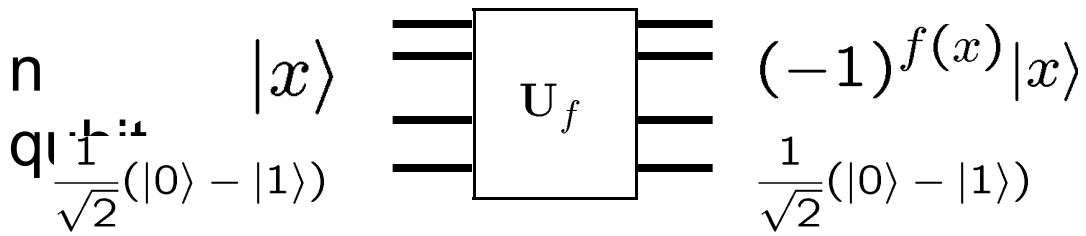
$$f(x_0) = 1$$
$$\forall x \neq x_0, f(x) = 0$$

Problem: find x_0 with as few queries as possible.



Grover's Algorithm

Use the black box in a particular way



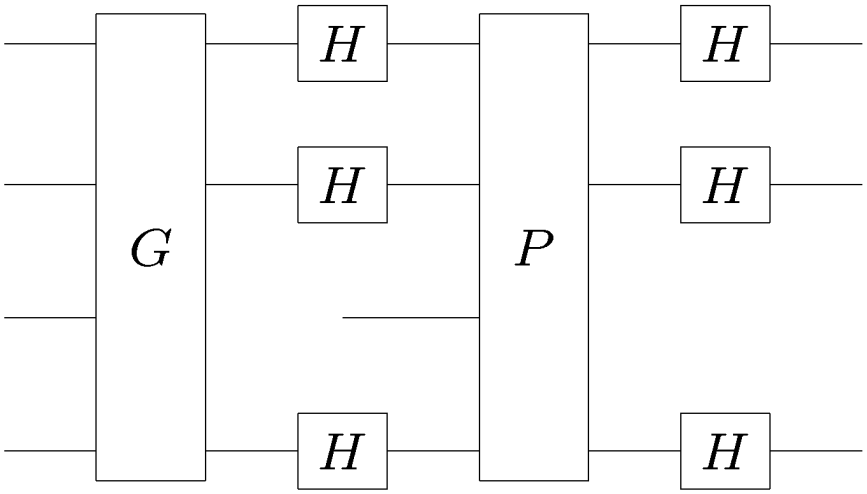
Grover oracle:

$$G = \sum_{x=0}^{2^n} (-1)^{f(x)} |x\rangle\langle x|$$

How to use Grover oracle to find $f(x_0) = 1$?

The Grover Iterate

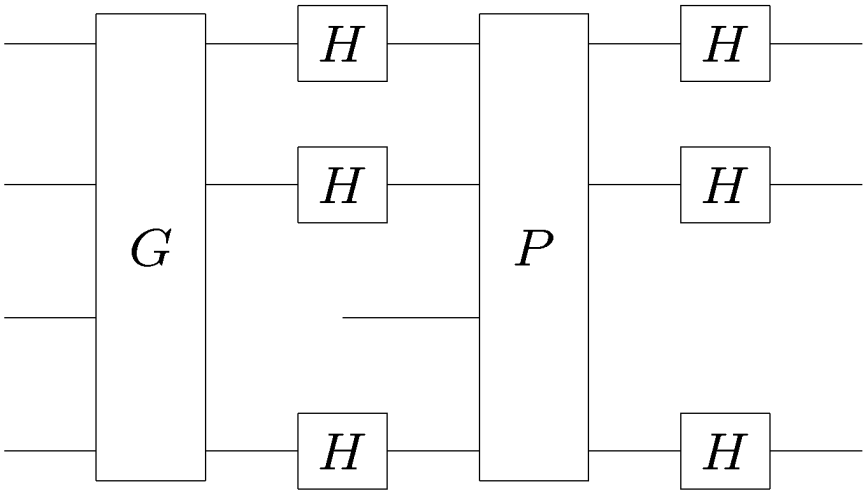
n
qubits



$$G = \sum_{x=0}^{2^n} (-1)^{f(x)} |x\rangle\langle x| \qquad P = \sum_{x=0}^{2^n} -(-1)^{\delta_{x,0}} |x\rangle\langle x|$$

The Grover Iterate

n
qubits



$$P = \sum_{x=0}^{2^n-1} -(-1)^{\delta_{x,0}}|x\rangle\langle x| = 2|0\rangle\langle 0| - I$$

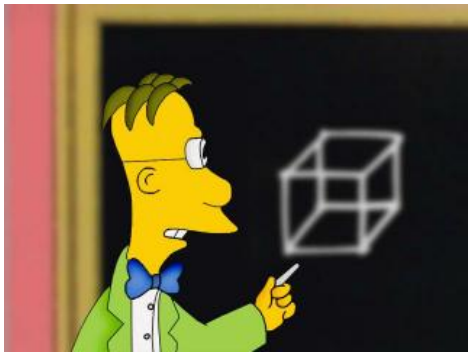
$$H^{\otimes n} P H^{\otimes n} = 2H^{\otimes n}|0\rangle\langle 0|H^{\otimes n} - I = 2|\psi\rangle\langle\psi| - I$$

$$|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$

Grover's iterate

$$R = (2|\psi\rangle\langle\psi| - I)G$$

The Grover Iterate in 2D



Two orthonormal vectors:

$$|\alpha\rangle = \frac{1}{\sqrt{2^n - 1}} \sum_{x=0|f(x)=0}^{2^n-1} |x\rangle \qquad |x_0\rangle$$

Express the equal superposition in terms of these:

$$|\psi\rangle = \sqrt{\frac{2^n - 1}{2^n}}|\alpha\rangle + \frac{1}{\sqrt{2^n}}|x_0\rangle$$

The Grover iterate will preserve this two dimensional subspace

$$G = \sum_{x=0}^{2^n-1} (-1)^{f(x)}|x\rangle\langle x| \qquad \begin{aligned} G|\alpha\rangle &= |\alpha\rangle \\ G|x_0\rangle &= -|x_0\rangle \end{aligned}$$

$$2|\psi\rangle\langle\psi| - I = 2\left[\left(\sqrt{\frac{2^n - 1}{2^n}}|\alpha\rangle + \frac{1}{\sqrt{2^n}}|x_0\rangle\right)\left(\sqrt{\frac{2^n - 1}{2^n}}\langle\alpha| + \frac{1}{\sqrt{2^n}}\langle x_0|\right)\right] - I$$

The Grover Iterate in 2D

Expressed over the two dimensional subspace:

$$G = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$2|\psi\rangle\langle\psi| - I = 2\left[\left(\sqrt{\frac{2^n-1}{2^n}}|\alpha\rangle + \frac{1}{\sqrt{2^n}}|x_0\rangle\right)\left(\sqrt{\frac{2^n-1}{2^n}}\langle\alpha| + \frac{1}{\sqrt{2^n}}\langle x_0|\right)\right] - I$$

$$2|\psi\rangle\langle\psi - I = \begin{bmatrix} 1 - \frac{1}{2^{n-1}} & \frac{\sqrt{2^n-1}}{2^{n-1}} \\ \frac{\sqrt{2^n-1}}{2^{n-1}} & \frac{1}{2^{n-1}} - 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ \sin\theta & -\cos\theta \end{bmatrix}$$

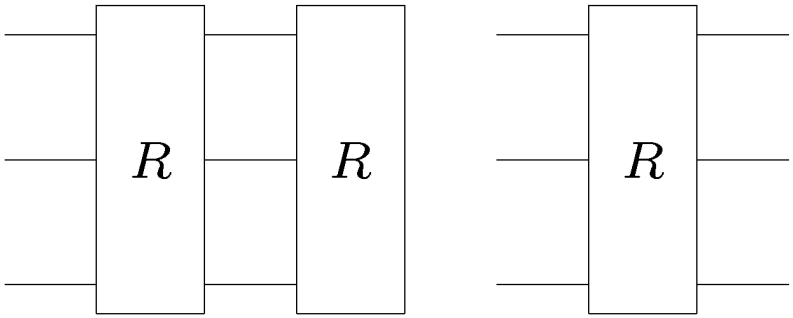
$$\sin\theta = \frac{\sqrt{2^n-1}}{2^{n-1}}$$

$$(2|\psi\rangle\langle\psi| - I)G = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Grover's iterate is just a rotation in this 2D space

Repeatedly Bang Your Head

Repeated application of the Grover iterate



$$R^k = [(2|\psi\rangle\langle\psi| - I)G]^k = \begin{bmatrix} \cos k\theta & -\sin k\theta \\ \sin k\theta & \cos k\theta \end{bmatrix}$$

Grover's algorithm:

1. start with $|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$
2. repeatedly apply Grover's iterate to rotate to near $|x_0\rangle$

Repeatedly Bang Your Head

$$R^k = [(2|\psi\rangle\langle\psi| - I)G]^k = \begin{bmatrix} \cos k\theta & -\sin k\theta \\ \sin k\theta & \cos k\theta \end{bmatrix}$$

$$|\psi\rangle = \sqrt{\frac{2^n - 1}{2^n}}|\alpha\rangle + \frac{1}{\sqrt{2^n}}|x_0\rangle$$

Large amplitude in "bad" part of Hilbert space

$$\sin\theta = \frac{\sqrt{2^n - 1}}{2^{n-1}} \quad \text{physicist: } \theta \approx \frac{\sqrt{2^n - 1}}{2^{n-1}} \approx \frac{2}{\sqrt{2^n}}$$

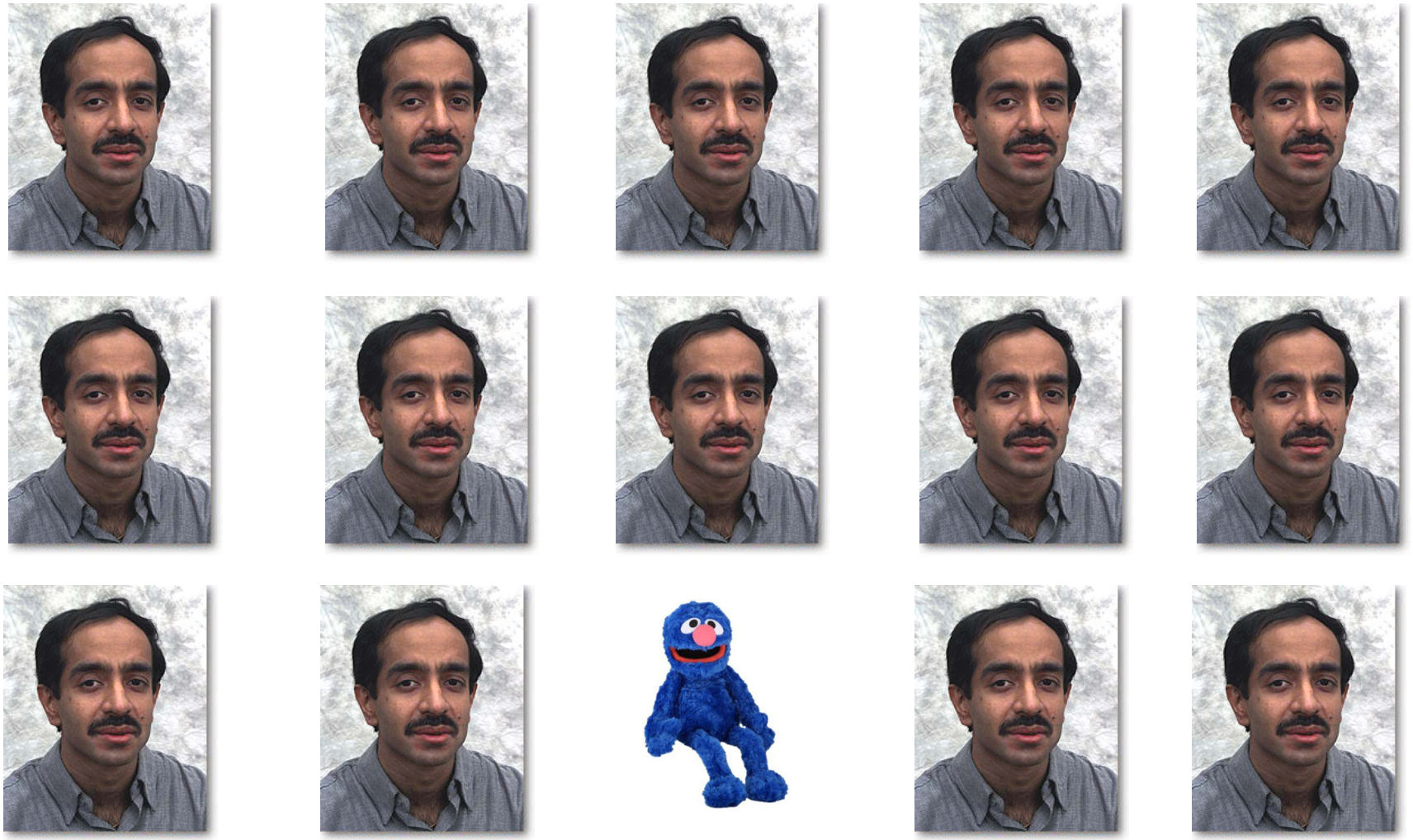
$$k\theta = \frac{\pi}{2} \quad \text{implies} \quad R^k = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \quad k \approx \frac{\pi}{4}\sqrt{2^n}$$

Application of the repeated iterate to initial state rotates it to $|x_0\rangle$

nearly all amplitude in

Grover's Algorithm

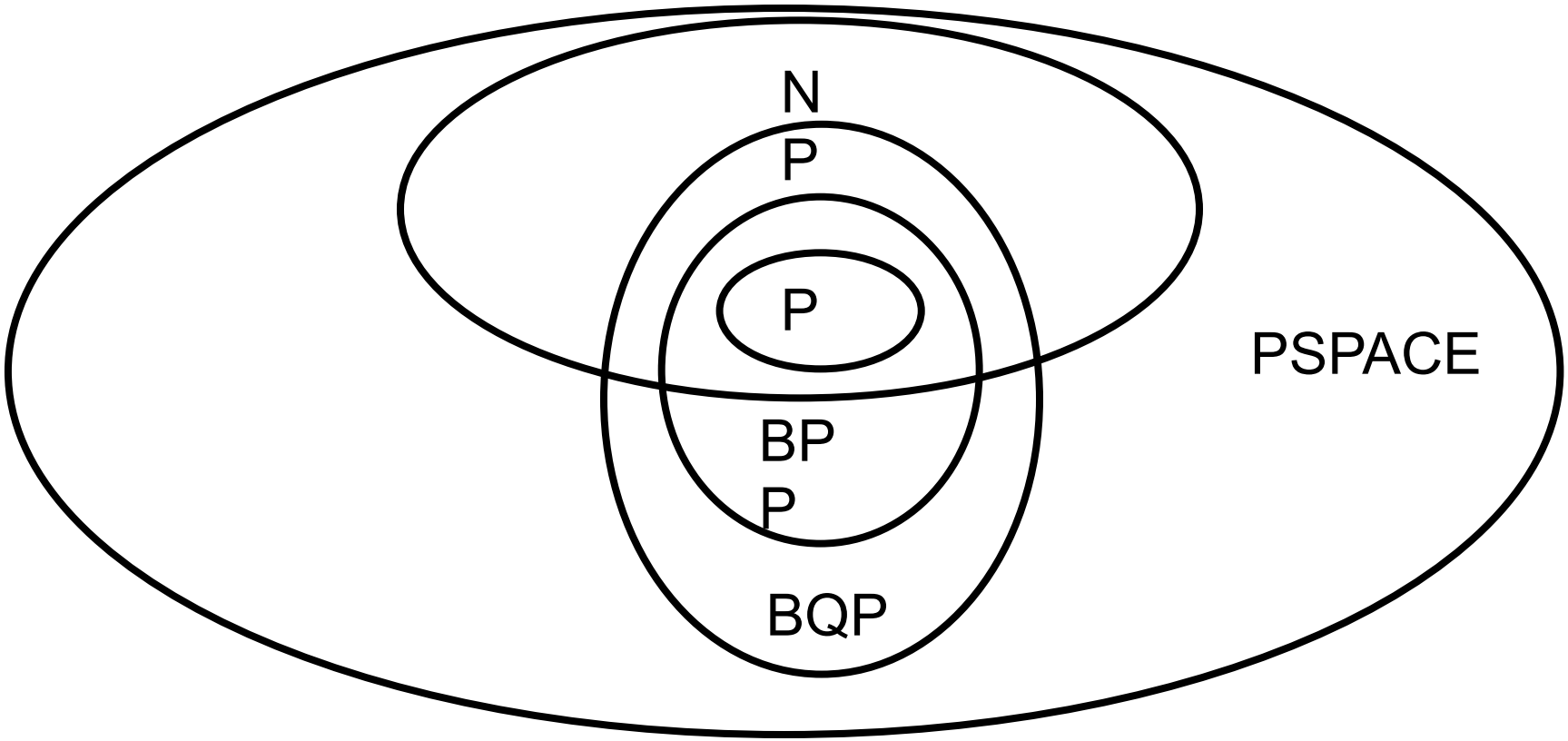
We have identified marked item using only $O(\sqrt{2^n})$ queries!



Quantum Complexity Theory

BPP (Bounded-error Probabilistic Polynomial time):
Error probability less than some fixed constant < ½

BQP (Bounded-error Quantum Polynomial time):
Error probability less than some fixed constant < ½



Quantum Algorithms

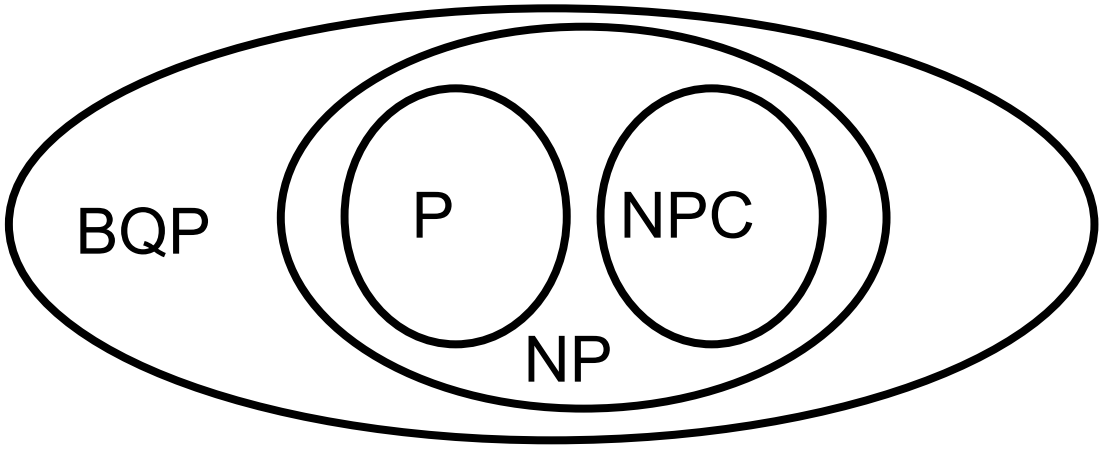
What else can quantum computers do?

- Factoring, discrete log [Shor 94]
- Unstructured search [Grover 96]
- Various hidden subgroup problems [Long List]
- Pell's equation [Hallgren 02]
- Hidden shift problems [van Dam, Hallgren, Ip 03]
- Graph traversal [CCDFGS 03]
- Spatial search [AA 03, CG 03/04, AKR 04]
- Element distinctness [Ambainis 03]
- Various graph problems [DHHM 04, MSS 03,…]
- Testing matrix multiplication [Buhrman,Špalek 04]
- $\mathbb{Z}_p^r \rtimes \mathbb{Z}_p$ hidden subgroup problem [Bacon, Childs, van Dam 05]
- Certain hidden shift problems [Childs, van Dam 05]

Quantum Algorithms

What else might quantum computer be able to do?

Not likely:

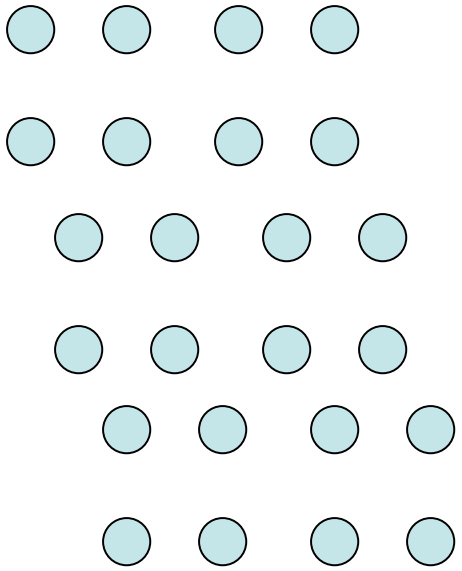


Interesting problems not NPC but possibly in BQP?

Graph isomorphism
Restricted shortest vector in a lattice problems
Finding Nash equilibrium
…

Quantum Simulation

Perhaps the least well studied and understood.



Simulating quantum many body systems is often computationally very difficult

Quantum computers allow one to perform these simulation without having to engineer entirely new physical systems.

Quantum materials? Understanding High-T Superconductors?